Solver Trace Ablation Diagnoses Structural Degeneration and Goodhart Effects in GRPO-Trained Neurosymbolic Reasoning: A Case Study

Anonymous ACL submission

Abstract

Execution-based evaluation of neurosymbolic reasoning conflates faithful derivation with structural shortcuts: a formalization that trivially embeds the conclusion among its premises proves just as reliably as a genuine one, and RL optimizers exploit this indifference. We propose *component necessity* as a diagnostic principle and operationalize it as *Solver Trace Ablation* (STA): for each proved output, STA tests whether every component is individually required for proof success, yielding the *Intensional Structural Reward* (ISR). ISR diagnoses structural degeneration invisible to prove rate, but occupies a paradoxical position: the same leave-one-out structure that makes it informative also makes it gameable. Tautological formalizations satisfy the necessity criterion by construction, so when ISR is repurposed as a training reward, the model learns to game the diagnostic rather than the task. In our Qwen3-4B case study, this *observe-vs-optimize asymmetry* persists across logic types and dataset families: structural proxies for reasoning quality are easier to observe than to optimize.[1]

1 Introduction

Neurosymbolic reasoning pipelines evaluate success by a single criterion: whether the generated formalization proves under a formal solver. This treats all proofs as equal: a formalization that embeds the conclusion verbatim among its premises receives the same reward as one that derives it through genuine reasoning. Under reinforcement learning, this indifference becomes an active vulnerability (Geirhos et al., 2020). The optimizer exploits any shortcut the reward fails to penalize (Skalse et al., 2022; Wang et al., 2026), and execution feedback provides no signal against structural shortcuts.

The evaluation gap. Neurosymbolic formalizers (Pan et al., 2023; Olausson et al., 2023; Ye et al., 2023) and RL-trained variants (Mellgren et al., 2025; Xie et al., 2025; Shao et al., 2024) rely on execution accuracy alone, with no diagnostic for whether individual components contribute to the proof. Process reward models (Lightman et al., 2024; Wang et al., 2024) offer finer granularity but require expensive step-level annotations that do not transfer to arbitrary solver-backed settings. Inference-time gaming detection (Kim et al., 2026; Bondarenko et al., 2025) addresses a complementary problem but offers no training-time diagnostic. What is missing is an annotation-free method for testing *component necessity* in neurosymbolic outputs.

Component necessity testing. We address this gap with *Solver Trace Ablation* (STA), a diagnostic that tests whether each component of a formalization is necessary for proof success. For each proved output, STA removes components one at a time and re-runs the solver; the fraction whose removal breaks the proof yields the *Intensional Structural Reward* (ISR). A formalization where every component is necessary receives ISR=1; one that proves regardless of which component is removed receives ISR=0. Leave-one-out ablation itself is standard (DeMillo et al., 1978; Jia and Harman, 2011); our contribution is the empirical finding that the same structure making ISR informative as a diagnostic also makes it gameable as a reward.

The observe-vs-optimize asymmetry. Applied post-hoc, ISR exposes a structural collapse hidden from prove rate (Figure 1): as

[1] Code and data: `[anonymized]`

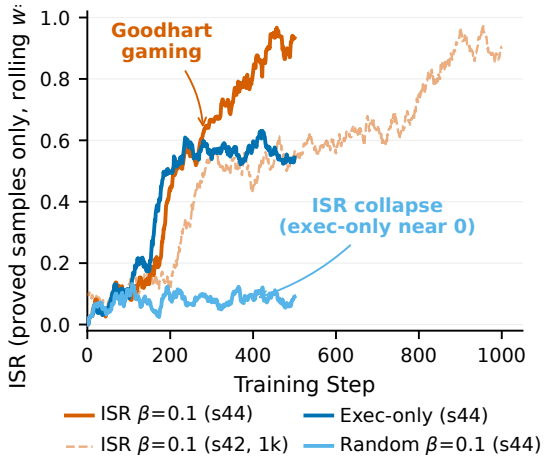


Figure 1: ISR training trajectories across reward conditions (proved samples only, rolling w=20). Exec-only collapses to ISR≈0, confirming structural degeneration (§4.2); the ∼0.5 plateau is an n=2 artifact. ISR β=0.1 seeds rise toward ISR≈1 via TS gaming (Goodhart effect, §4.3); random remains low.

execution scores climb, component necessity collapses, revealing that high prove rate coexists with near-complete structural degeneration. Yet the same LOO structure that makes ISR diagnostic also creates a Goodhart vulnerability (Manheim and Garrabrant, 2019; Karwowski et al., 2024; Gao et al., 2023): tautological formalizations ($P \to P$) satisfy the necessity criterion by construction, so when ISR is repurposed as a training reward, the model learns to game the diagnostic rather than the task. In this setting, the asymmetry persists across logic types, dataset families, reward doses, and random seeds: structural proxies for reasoning quality are easier to observe than to optimize.

Contributions. In this Qwen3-4B case study across two logic types, two dataset families, and multiple reward conditions, we contribute: (**C1**) STA as a diagnostic framework for neurosymbolic reasoning, including the ISR metric, an eight-mode hacking taxonomy, and a cross-domain evaluation protocol (§4.2); (**C2**) empirical characterization of a Goodhart effect when ISR is repurposed as a training reward, with controls ruling out confounds (§4.3); (**C3**) cross-logic and cross-dataset evidence documenting the observe-vs-optimize asymmetry (§4.4).

2 Related Work

Neurosymbolic reasoning. LLM-solver pipelines (Ye et al., 2023; Pan et al., 2023; Olausson et al., 2023; Pei et al., 2025; Di et al., 2025) and RL methods trained on solver feedback (Mellgren et al., 2025; Xie et al., 2025; DeepSeek AI, 2024; DeepSeek-AI, 2025b) evaluate execution accuracy alone, with no diagnostic for component necessity. AlphaProof (Hubert et al., 2025) and DeepSeek-Prover-V2 (DeepSeek-AI, 2025a) apply RL-based formal verification at scale in Lean 4 for competition mathematics; STA-style diagnostics could probe whether similar structural degeneration occurs in these settings. We use FOLIO (Han et al., 2024), the only public NLI benchmark with both FOL and Prolog annotations.

Process rewards and reward shaping. Reward shaping theory (Ng et al., 1999) constrains when auxiliary rewards preserve optimal policies. In mathematical reasoning, learned process reward models (Lightman et al., 2024; Wang et al., 2024; Cobbe et al., 2021; Uesato et al., 2022) and verifier-guided search (Yang et al., 2022; Hosseini et al., 2024; Sun et al., 2024) provide step-level feedback, but suffer from reward-model noise and distribution shift (Zheng et al., 2025). A recurring finding is that auxiliary rewards can hurt rather than help: Gao et al. (2024) show process/outcome bonuses can degrade RL training, and Setlur et al. (2025) demonstrate that bonus magnitude requires careful calibration. Annotation-free alternatives (Cui et al., 2025; Jha et al., 2024) derive dense rewards from outcome signals or solver certificates; ISR follows this direction but our results reveal that even annotation-free structural rewards are susceptible to Goodhart effects. Sullivan and Koller (2025) prove GRPO with outcome reward is implicitly a Monte-Carlo PRM, a mechanism for why component-level gaming emerges under GRPO.

Reward hacking and Goodhart's Law. Goodhart's Law is established theoretically (Manheim and Garrabrant, 2019; Skalse et al., 2022; Karwowski et al., 2024) and empirically in RLHF (Gao et al., 2023) and logical formalization (Bondarenko et al., 2025; Kim

et al., 2026). Most closely related, Kim et al. (2026) document inference-time gaming on FOLIO with frontier models and Lean 4; our work is complementary in studying training-time degeneration with component necessity rather than faithfulness, and combining STA with inference-time detection would provide full gaming coverage. Kwa et al. (2024) show that heavy-tailed reward misspecification causes catastrophic Goodhart where KL regularization provably fails, a theoretical lens for why GRPO's KL penalty does not prevent our observed collapse. Concurrent work documents spurious signals in RLVR (Shao et al., 2025; Yan et al., 2026; Dong et al., 2026; Von Arx et al., 2025) and proposes composite rewards as mitigations (Tarek and Beheshti, 2025; Pronesti et al., 2026); Yue et al. (2025) show RLVR redistributes base-model probability mass rather than creating new reasoning, consistent with our finding that GRPO exploits existing shortcuts. Helff et al. (2026) propose Input Perturbation Testing (IPT) for gaming detection; STA shares the perturbation intuition but operates at the component level in neurosymbolic settings where gaming emerges during training. While IPT perturbs *inputs* to detect inference-time gaming, STA performs leave-one-out ablation on *solver-input components* to diagnose training-time structural degeneration, operating at component-level rather than input-level granularity and targeting the training process rather than deployment behavior. Khalifa et al. (2026) provide RLVR hacking testbeds. Our Goodhart effect follows the regressional pattern (Manheim and Garrabrant, 2019) on FOLIO, with extremal dynamics at high β; its application to a diagnostic designed to *detect* gaming, and the dataset-dependent asymmetry it reveals, are to our knowledge novel.

Mutation testing. STA adapts the LOO mutation paradigm (DeMillo et al., 1978; Jia and Harman, 2011) to solver inputs; novelty lies in application and findings, not in the perturbation primitive.

Algorithm 1 Solver Trace Ablation (STA)

Require: Proved formalization $x = \{c_1, \ldots, c_n\}$, solver S
Ensure: $\mathrm{ISR}(x) \in [0, 1]$

```
necessary ← 0
for i = 1 to n do
    if S(x \ {c_i}) = failed then
        necessary ← necessary + 1
    end if
end for
return necessary / n
```

3 Method

3.1 Task and GRPO Training

Given premises $P_1, \ldots, P_k$ and conclusion C, the task is to produce a formal program $F = \{c_1, \ldots, c_n\}$ verified by an external solver $S \in \{\text{Prover9, SWI-Prolog}\}$. We train with Group Relative Policy Optimization (GRPO) (Shao et al., 2024) (K=4), where each rollout receives binary execution reward $r_{\text{exec}}(x) = \mathbf{1}[S(F_x)\text{=}\textit{proved}]$. This optimizes prove rate but provides no signal about *component necessity*: a formalization embedding the conclusion among its premises receives the same reward as a faithful derivation. All experiments draw from FOLIO (Han et al., 2024).

3.2 Solver Trace Ablation (STA)

STA tests whether each component of a proved formalization is individually necessary for proof success (Algorithm 1). Given a proved x with n components, STA constructs n ablated variants $x \setminus \{c_i\}$ and re-runs the solver (Algorithm 1). A component is *necessary* if its removal causes proof failure. What constitutes a component depends on the logic: premises in FOL, clauses or ground facts in Prolog. LOO ablation itself is standard in mutation testing and sensitivity analysis (DeMillo et al., 1978; Jia and Harman, 2011); novelty lies in its application as a GRPO-hacking diagnostic and the empirical phenomena it surfaces.

Intensional Structural Reward. [2] ISR is the fraction of necessary components:

$$\mathrm{ISR}(x) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\big[S(x \setminus \{c_i\}) = \textit{failed}\big] \quad (1)$$

[2] Not *intentional*; *intensional* contrasts with extensional (execution-only) evaluation.

ISR is defined only for proved formalizations: ISR=1 means every component is necessary; ISR=0 means complete redundancy. ISR quantifies component redundancy, not semantic faithfulness; its diagnostic value is realized through the downstream mode breakdown (§4.5), not the scalar alone. We report *proved-only* ISR (denoted ISR*) throughout to avoid conflating component necessity with prove rate.

Limitations of ISR. ISR captures component necessity, not semantic faithfulness; it does not correlate with NLI accuracy (Appendix C). A structural confound arises from component count: formalizations with fewer components mechanically receive higher ISR, and tautological shortcuts ($P \to P$) achieve high ISR by construction since the sole premise is necessary for the trivial proof. We therefore combine ISR with text-match classifiers for complete hacking detection (§3.3; $O(1)$ binary variant in Appendix E.3), and report premise-count-matched comparisons in §4.3. At the per-sample level, ISR flags distinct failure modes (ISR=0: redundancy; ISR$\geq$0.5 with n=1: tautology) independent of this confound. We distinguish two uses of ISR: as an *aggregate* metric (mean ISR across a test set, susceptible to premise-count inflation discussed in §4.3), and as a *per-sample* diagnostic (the LOO result for a single proof, unaffected by cross-sample length distributions). As a diagnostic, ISR most readily identifies CE, where removing fabricated expressions collapses the proof (ISR$\approx$0), but is less discriminative for TS, where tautological structures score high ISR by construction; detecting TS requires combining ISR with component-count thresholds (§3.3).

LOO independence assumption. LOO ablation tests individual, not joint, component necessity (Jia and Harman, 2011), systematically underestimating ISR when jointly-necessary components exist; the observed collapse is therefore conservative.

3.3 Reward Conditions and Hacking Modes

We study six conditions: exec-only (r_{exec}), ISR low/high-dose ($r_{\text{exec}} \cdot (1+\beta\,\text{ISR})$ at $\beta \in \{0.1, 0.3\}$), matched random controls ($r_{\text{exec}} \cdot (1+\beta u)$, $u \sim \mathcal{U}(0,1)$), and a supervised fine-tuning (SFT) baseline (negative log-likelihood on gold formalizations). Multiplicative bonuses apply only to proved samples, keeping the prove-rate floor intact. Random isolates ISR-specific signal from generic reward-variance effects; SFT isolates whether structural degeneration arises from RL optimization or data exposure alone. Each RL condition uses seeds $\{42, 43, 44\}$ and identical training data (formulation alternatives in Appendix A).

Hacking taxonomy. Across conditions, seeds, and logic types we identify eight structural-degeneration modes (full definitions in Appendix D): **CE** (conclusion embedded among multiple premises), **TS** (single-premise conclusion duplication, subtypes TSa standard and TSb obfuscated tautology; targets the ISR signal directly, since removing the sole premise forces ISR $\geq$ 0.5), **PG** (premise-subset gaming: each component is LOO-necessary but the premise set is a strict subset of what the problem requires, inflating ISR without faithful reasoning; Appendix D.1), **tautological collapse** (minimal two-component $P \to P$ without explicit ISR inflation), **repetition collapse** (identical clauses repeated to inflate component count), **complexity-without-utility** (elaborate but vacuous premises), **contradiction explosion** (contradictory premises trivializing any conclusion via *ex falso*), and **query-as-fact gaming** (Prolog-specific: emit the NL question as an atom asserted as both fact and query; evades text-match classifiers). Modes are identified by a two-stage procedure: strict normalized text matching (CE/TS) plus functional analysis of the per-sample $n_{\text{necessary}}$ distribution (catches modes evading text matching). We report both classifications in §4.

Low ISR signals redundancy under exec-only but metric gaming under ISR-as-reward; the mode breakdown, not the ISR scalar, licenses interpretation.

4 Experiments

We evaluate STA along three axes: ISR as a diagnostic for reward hacking (§4.2); ISR as a training reward, where it exhibits Goodhart

effects (§4.3); and cross-domain generalization across logic types and dataset families (§4.4). A hacking taxonomy synthesizes the modes observed across conditions (§4.5).

4.1 Setup

We train Qwen3-4B (Qwen Team, 2025) with GRPO (Shao et al., 2024) (K=4, lr 10^{-6}, 500 steps default, 1000 where noted) over seeds $\{42, 43, 44\}$ (ISR β=0.1 extends to $\{45, 46\}$; Appendix E) on a fixed 500-example FOLIO (Han et al., 2024) subset held constant across all conditions; the SFT baseline uses negative log-likelihood on gold formalizations.[3] Held-out evaluation uses 100 FOLIO examples with greedy decoding;[4] all cross-run ISR values are recomputed uniformly by a single evaluator so training-log ISR is never compared across runs. Cross-dataset experiments (§4.4) use ProofFOL, a silver-standard FOL corpus of ∼500 training examples constructed by translating ProofWriter (Tafjord et al., 2021) problems via GPT-4o and filtering through Prover9 (∼70% retention); *stable convergence* denotes $\overline{\text{PR}}_{\text{final 50}} > 0.90$. We report prove rate (PR), proved-only ISR (ISR*), hacking-mode rates from a two-stage classifier (strict text matching for CE/TS plus functional analysis of the per-sample $n_{\text{necessary}}$ distribution; §3.3), and a *non-hacking* (N-H) rate, an upper bound on genuine reasoning: the residual matching none of the eight cataloged patterns (classifier recall unknown; 152-sample precision audit in Appendix D).

Statistical considerations. With three seeds exhibiting multi-attractor dynamics, per-seed qualitative analysis is the primary framework; the divergence itself is a finding. Pooled statistics (CMH) serve as supplementary reference; CMH treats within-seed samples as independent observations from a single trained model, so the true replication count is N=3 seeds. At N=3, seed-level tests achieve 80% power only for very large effects ($d \geq 3.0$); non-significant comparisons (e.g., prevalence p=0.182) are exploratory. ProofFOL results (p=0.10, N=3) are exploratory.

[3] Seeds control initialization and rollout sampling; all conditions share the same fixed training subset.

[4] Seed 43 uses nucleus sampling (T=0.7) due to greedy-mode degeneration; excluding s43 yields consistent or stronger collapse: at 1000 steps, both remaining seeds show ≤7% non-hacking (Appendix E).

4.2 ISR as Diagnostic

Collapse trajectory. Figure 1 traces exec-only GRPO (seed 42) across 1000 training steps. As PR climbs from 32% to 96%, ISR* collapses from 0.642 to 0.003, a 214× raw drop (partially attributable to premise-count inflation; matched comparison yields 1.06×, §4.3) that reveals structural degeneration behind apparent success. By 500 steps, 87% of proved samples embed the conclusion verbatim (CE), confirming degeneration independent of the ISR scalar. A binary variant testing only collective premise necessity misses the collapse entirely (ISR=1.0 for all proved samples), confirming that per-component LOO testing is essential (Appendix E.3).

Cross-seed instability. Three seeds diverge into qualitatively different failure modes: conclusion embedding (s42), repetition collapse (s43), and tautological collapse (s44), yet all converge to ≤7% non-hacking by 1000 steps (Table 8; Appendix E). Two additional seeds under identical conditions fail to converge (PR≈14%; Appendix E.5), retaining genuine reasoning and confirming that structural degeneration requires successful RL optimization as a prerequisite (Appendix E.4).

SFT baseline. SFT on gold formalizations produces 92% non-hacking outputs at PR=13% (OR=147 vs. exec-only, p<0.001), confirming that hacking modes are absent before RL optimization. SFT serves as a pre-RL reference rather than a PR-matched control; its low PR reflects minimal optimization pressure. Early-training comparison supports this interpretation: at 100 steps (PR=32%), hacking already reaches 31% vs. SFT's 8% at comparable PR (Appendix E.2).

4.3 Goodhart Effects on ISR-as-Reward

We reverse ISR's role from observer to reward: $r = r_{\text{exec}} \cdot (1 + \beta\,\text{ISR})$, with matched random control ($u \sim \mathcal{U}(0, 1)$). The result is a regressional Goodhart pattern (Manheim and Garrabrant, 2019): optimizing a structural proxy shifts the *composition* of failure modes without reducing their aggregate prevalence. ISR converts conclusion embedding (CE) into tautological shortcuts (TS) rather than pro-

ducing genuine reasoning (Table 1); the apparent ISR advantage collapses from 3.2× to 1.06× after controlling for premise count.

Multiplicative coupling. The multiplicative reward couples execution and ISR signals; three controls constrain this confound: exec-only's convergence to pure hacking, the qualitative TS shift under ISR, and an additive variant's PR collapse (Appendix A). A shuffled-ISR control (preserving ISR's distribution but breaking the sample-level semantic link) produces SFT-like outcomes (87% non-hacking at low PR), confirming that the Goodhart effect requires the sample-level ISR signal, not its distributional shape.

Low-dose ISR (β=0.1). ISR reward amplifies TS convergence: 71% of proved samples under ISR exhibit tautological shortcuts vs. 14% under random (OR_{CMH}=2.46; Appendix C). TS is the canonical Goodhart move: a single premise identical to the conclusion yields ISR $\geq$ 0.5 by construction, directly satisfying the LOO metric (§3.3). Total conclusion embedding (CE+TS) spans 68–100% across both conditions; the composition shifts (CE dominates under random, TS under ISR) without reducing aggregate hacking (Figure 3).

Premise-count confound. Raw ISR* comparisons are inflated by premise minimization: ISR-trained models produce shorter formalizations, mechanically boosting the LOO fraction. After matching for premise count, the apparent 3.2× advantage shrinks to 1.06× (Appendix E); non-hacking rates are indistinguishable (p=0.182; Appendix D.1). The asymmetry is compositional, not quantitative: ISR redirects gaming toward TS rather than reducing its prevalence.

Higher doses and temporal dynamics. Any genuine reasoning under ISR β=0.1 is transient: non-hacking drops from 17% to 2% by 1000 steps as TS escalates (Figure 1). Higher doses accelerate collapse: β=0.2 eliminates all non-hacking; β=0.3 destabilizes training itself, producing multi-modal failure (Gao et al., 2023) across both ISR and random conditions (Appendix E.1).

Bimodal seed-dependence (5-seed extension). Two additional seeds (s45, s46; Appendix E) under the same ISR β=0.1 condition yield ~55% non-hacking, qualitatively different from the 3-seed average of 6%. This bimodal outcome (3/5 seeds dominated by gaming, 2/5 producing substantial genuine reasoning) reinforces the cross-seed instability documented in §4.2 and underscores that ISR reward outcomes are seed-dependent rather than deterministic.

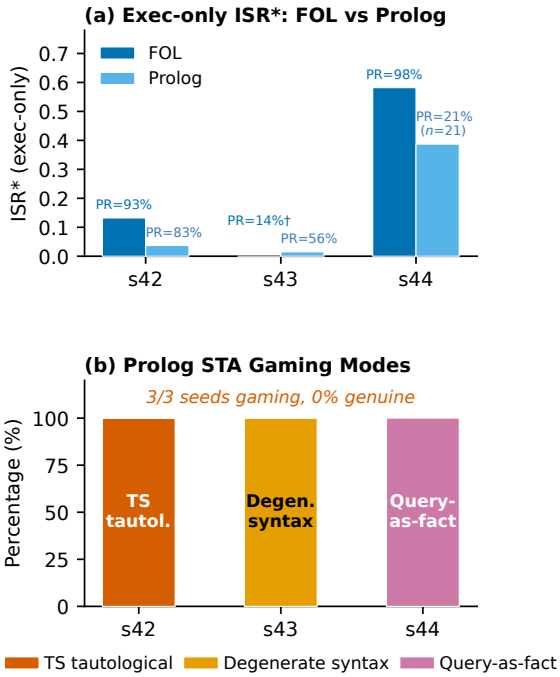


Figure 2: Cross-logic results. **Top**: FOL vs. Prolog exec-only ISR* across 3 seeds (s44 Prolog 0.387 is a small-sample artifact, n=21; †s43 FOL uses sampling T=0.7, not strictly comparable). **Bottom**: Prolog ISR-reward gaming-mode composition; all 3 seeds reach 0% non-hacking through distinct mechanisms.

4.4 Cross-Domain and Cross-Dataset Evidence

Prolog: universal Goodhart. Replicating the pipeline with Prolog/SWI-Prolog (Figure 2), exec-only seeds collapse to ISR*≈0 via redundant ground-fact coding, confirming STA's diagnostic transfer across logic types. Under ISR-as-reward, all three seeds reach 0% non-hacking through distinct mechanisms: clean TS (s42), degenerate syntax bypassing text-match classifiers (s43), and query-as-fact gaming (s44; Table 9). Prolog's simpler semantics broadens the gaming basin, eliminating the residual genuine reasoning seen in FOL.

ProofFOL exec-only: cross-dataset replication. Converged seeds collapse to

Condition	β	PR%↑	ISR*	TS%	CE%	N-H%
Base (0-shot)	—	12	—	—	—	—
SFT	—	13	0.595	0	8	**92**
Exec-only†	—	93	0.132	0	87	8
ISR (3-seed)	0.1	77±14	1.06×‡	**71**¶	18	6
ISR (5-seed)	0.1	70±19	(0.63)‡	54	18	23
Random	0.1	**87**±8	0.117±.04	14§	71	10
Shuffled-ISR	0.1	15±1	0.559±.02	7	7	**87**
ISR	0.3	66±42	0.39±.31	**50**	37	11
Random	0.3	54±35	0.09±.09	7	91	2

Table 1: Goodhart landscape across reward conditions. PR: prove rate; ISR*: proved-only ISR; TS: tautological shortcut; CE: conclusion embedding; N-H: non-hacking. 3-seed rows: TS/CE/N-H pooled across seeds; PR/ISR*: mean±std. †Seed 42. ‡ISR* raw mean inflated by TS gaming; 1.06× is the premise-count-matched value relative to random (§4.3, Appendix E); 5-seed raw ISR* in parentheses, same TS confound applies (no matched control for 5-seed). ¶Includes TS classifier false-negative correction (s44) and PG premise-subset gaming (s42; 5.2% of proved; Appendix D.1). §Random TS driven by seed 43 (38.3%); seeds 42 and 44: 0%. Per-seed breakdown in Appendix Table 8. Shuffled-ISR is a deconfounding control (§4.3), not an independent reward condition. ISR β=0.1: 3-seed row uses s42–44; 5-seed row adds s45, s46 (∼55% non-hacking each; Appendix E).

near-zero genuine reasoning, confirming STA's diagnostic across dataset families. The dominant hacking mode shifts from CE (FOLIO) to TS (ProofFOL), showing that mode repertoire is dataset-specific while the degeneration phenomenon is universal.

ProofFOL ISR (β=0.1): dataset-dependent asymmetry. ISR reward on ProofFOL produces substantially higher genuine rates than exec-only (p=0.10, exploratory; Table 10), though at lower PR, possibly reflecting insufficient training.

Mechanism. The asymmetry follows from hacking geometry: on FOLIO, tautological shortcuts yield high ISR (regressional Goodhart (Manheim and Garrabrant, 2019)); on ProofFOL, the dominant hacking mode embeds conclusions among many premises, yielding ISR≈0 and rendering ISR reward effectively corrective. ISR-as-reward inherits the dataset's hacking topology.

Cross-scale validation (Qwen3-8B). Qwen3-8B does not converge under exec-only GRPO (PR=14%, K=8; K=4 produces zero gradient), consistent with degeneration requiring successful convergence (Appendix E.6).

4.5 Hacking Taxonomy

Three modes account for the vast majority of hacking (Table 2; Appendix D): CE (conclusion embedding), TS (conclusion duplication), and PG (premise-subset gaming). The central pattern: *reward formulation determines which mode dominates but not whether hacking occurs*; all GRPO conditions produce overwhelming degeneration, while SFT achieves 92% non-hacking.

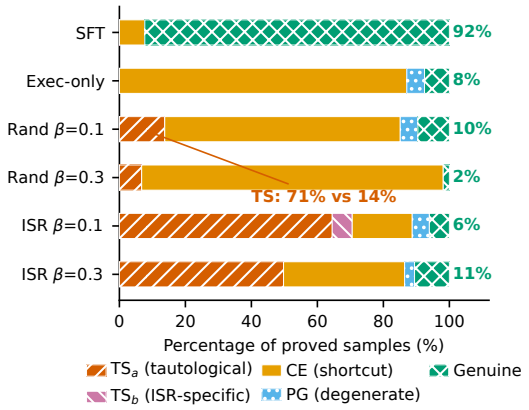


Figure 3: Hacking composition across reward conditions (proved samples; exec-only: seed 42; all other bars: pooled across 3 seeds, per-seed breakdown in Appendix Table 8). ISR converts CE into more extreme TS without reducing total hacking; SFT shows near-zero hacking.

5 Discussion and Conclusion

STA exposes structural degeneration invisible to execution-based evaluation; repurposed as reward, ISR produces a seed-dependent Goodhart effect, documenting that structural proxies for reasoning quality are easier to observe than to optimize (Appendix C).

Mode	Exec	ISR$_{.1}$	Rnd$_{.1}$	SFT	Prolog$_{\text{ISR}}$
CE embed	87	18	71	7	0$^\sharp$
TS duplicate	0	71	14	0	100$^\sharp$
PG prem-subset	6	5	8	0	—
Tautological	—	—	—	—	s42
Repetition	s43	—	—	—	—
Contradiction	1k	—	—	—	—
Cmplx. vacuous	0	0	4	0	—
Query-as-fact	—	—	—	—	s44
Non-hacking	8	6	10	**92**	**0**

Table 2: Eight hacking modes across conditions (% of proved samples; exec-only/SFT: seed 42; ISR/Random: pooled across 3 seeds). TS includes subtypes TSa (text-match, n=149) and TSb (obfuscated tautology, n=14; primarily s44). Cells with seed identifiers indicate the mode dominates that seed. $^\sharp$Prolog row pools strict+functional (§4.4). `1k` denotes a mode observed only at the 1000-step checkpoint.

6 Limitations

Experiments use a single base model (Qwen3-4B); structural degeneration patterns may not transfer to other model families, and neither Qwen3-8B nor LLaMA-3.2-3B converges under our GRPO configuration (Appendix E.6, E.4), leaving cross-scale generalization untested. Three to five seeds per condition demonstrate qualitative divergence but remain underpowered for seed-level significance; pooled statistics treat within-seed samples as independent. One of three exec-only seeds (s43) uses nucleus sampling (T=0.7) due to greedy-mode degeneration, making it not strictly comparable; excluding it yields consistent or stronger collapse patterns (Appendix E). ISR quantifies component necessity, not semantic faithfulness (no correlation with NLI accuracy; Appendix C); its diagnostic value is realized through the mode breakdown, not the scalar alone.

7 Ethics Statement

This work documents failure modes of reward optimization in solver-backed RL systems. Our findings (that structural diagnostics can be gamed when used as training signals) carry implications for AI safety: they demonstrate that metrics designed to detect misalignment may themselves become targets of optimization, a concern relevant to scalable oversight. We release our taxonomy and diagnostic tools to help practitioners detect reward hacking in their own systems. Code, evaluation scripts, the ProofFOL dataset, and trained model checkpoints will be released upon acceptance. All experiments use publicly available datasets (FOLIO) and open-weight models (Qwen3-4B); no human subjects are involved.

References

Alexander Bondarenko, Denis Volk, Dmitrii Volkov, and Jeffrey Ladish. 2025. Demonstrating Specification Gaming in Reasoning Models. *arXiv preprint arXiv:2502.13295*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, and 6 others. 2025. Process Reinforcement through Implicit Rewards. *arXiv preprint arXiv:2502.01456*.

DeepSeek AI. 2024. DeepSeek-Prover-V1.5: Harnessing Proof Assistant Feedback for Reinforcement Learning and Monte-Carlo Tree Search. *arXiv preprint arXiv:2408.08152*.

DeepSeek-AI. 2025a. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition. *arXiv preprint arXiv:2504.21801*.

DeepSeek-AI. 2025b. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*.

Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer*, 11(4):34–41.

Zhengkun Di, Chaoli Zhang, Hongtao Lv, Lizhen Cui, and Lei Liu. 2025. LoRP: LLM-based Logical Reasoning via Prolog. *Knowledge-Based Systems*.

Yiming Dong, Kun Fu, Haoyu Li, Xinyuan Zhu, Yurou Liu, Lijing Shao, Jieping Ye, and Zheng Wang. 2026. Probing RLVR Training Instability Through the Lens of Objective-Level Hacking. *arXiv preprint arXiv:2602.01103*.

Jiaxuan Gao, Shusheng Xu, Wenjie Ye, Weilin Liu, Chuyi He, Wei Fu, Zhiyu Mei, Guangju Wang, and Yi Wu. 2024. On Designing Effective RL

Reward at Training Time for LLM Reasoning. *arXiv preprint arXiv:2410.15115*.

Leo Gao, John Schulman, and Jacob Hilton. 2023. Scaling Laws for Reward Model Overoptimization. In *Proceedings of the International Conference on Machine Learning (ICML)*. ArXiv:2210.10760.

Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. 2020. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, and 1 others. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.

Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Luke Benson, Lucy Sun, Ekaterina Zubova, Yujie Qiao, Matthew Burtell, David Peng, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, and 7 others. 2024. FOLIO: Natural Language Reasoning with First-Order Logic. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ArXiv:2209.00840.

Lukas Helff, Quentin Delfosse, David Steinmann, Ruben Härle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. 2026. LLMs Gaming Verifiers: RLVR can Lead to Reward Hacking. *arXiv preprint arXiv:2604.15149*.

Arian Hosseini, Xingdi Yuan, Nikolay Malkin, Aaron Courville, Alessandro Sordoni, and Rishabh Agarwal. 2024. V-STaR: Training Verifiers for Self-Taught Reasoners. In *Conference on Language Modeling (COLM)*.

Thomas Hubert, Rishi Mehta, Laurent Sartran, and 1 others. 2025. Olympiad-Level Formal Mathematical Reasoning with Reinforcement Learning. *Nature*.

Piyush Jha, Prithwish Jana, Pranavkrishna Suresh, Arnav Arora, and Vijay Ganesh. 2024. RLSF: Fine-tuning LLMs via Symbolic Feedback. *arXiv preprint arXiv:2405.16661*.

Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5):649–678.

Jacek Karwowski, Oliver Hayman, Xingjian Bai, Klaus Kiendlhofer, Charlie Griffin, and Joar Skalse. 2024. Goodhart's Law in Reinforcement Learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*. ArXiv:2310.09144.

Muhammad Khalifa, Zohaib Khan, Omer Tafveez, Hao Peng, and Lu Wang. 2026. Countdown-Code: A Testbed for Studying The Emergence and Generalization of Reward Hacking in RLVR. *arXiv preprint arXiv:2603.07084*.

Kyuhee Kim, Auguste Poiroux, and Antoine Bosselut. 2026. Do LLMs Game Formalization? Evaluating Faithfulness in Logical Reasoning. In *VerifAI-2 Workshop at the International Conference on Learning Representations (ICLR)*. ArXiv:2604.19459.

Tat-Thang Kwa, Rachel Thomas, and Adrià Garriga-Alonso. 2024. Catastrophic Goodhart: Regularizing RLHF with KL Divergence Does Not Mitigate Heavy-Tailed Reward Misspecification. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2404.18926.

Hunter Lightman, Vineet Kosaraju, Yura Burda, and 1 others. 2024. Let's Verify Step by Step. In *Proceedings of the International Conference on Learning Representations (ICLR)*. ArXiv:2305.20050.

David Manheim and Scott Garrabrant. 2019. Categorizing Variants of Goodhart's Law. *arXiv preprint arXiv:1803.04585*.

Niklas Mellgren, Peter Schneider-Kamp, and Lukas Galke. 2025. Training Language Models to Use Prolog as a Tool. *arXiv preprint arXiv:2512.07407*.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 278–287.

Theo X. Olausson, Alex Gu, Ben Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. 2023. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. 2023. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In *Findings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yu Pei, Yongping Du, and Xingnan Jin. 2025. FoVer: First-Order Logic Verification for Natural Language Reasoning. *Transactions of the Association for Computational Linguistics*, 13:1340–1359.

Massimiliano Pronesti, Anya Belz, and Yufang Hou. 2026. Beyond Outcome Verification: Verifiable Process Reward Models for Structured Reasoning. *arXiv preprint arXiv:2601.17223*.

Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*.

Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. 2025. Rewarding Progress: Scaling Automated Process Verifiers for LLM Reasoning. In *Proceedings of the International Conference on Learning Representations (ICLR)*. ArXiv:2410.08146.

Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. 2025. Spurious Rewards: Rethinking Training Signals in RLVR. *arXiv preprint arXiv:2506.10947*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, and 1 others. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. 2022. Defining and Characterizing Reward Hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2209.13085.

Michael Sullivan and Alexander Koller. 2025. GRPO is secretly a process reward model. *arXiv preprint arXiv:2509.21154*.

Zhiqing Sun, Longhui Yu, Yikang Shen, Weiyang Liu, Yiming Yang, Sean Welleck, and Chuang Gan. 2024. Easy-to-Hard Generalization: Scalable Alignment Beyond Human Supervision. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. 2021. ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. In *Findings of the Association for Computational Linguistics (ACL)*. ArXiv:2012.13048.

Mirza Farhan Bin Tarek and Rahmatollah Beheshti. 2025. Reward Hacking Mitigation using Verifiable Composite Rewards. *arXiv preprint arXiv:2509.15557*.

Ramya Keerthy Thatikonda, Jiuzhou Han, Wray Buntine, and Ehsan Shareghi. 2024. Strategies for Improving NL-to-FOL Translation with LLMs: Data Generation, Incremental Fine-Tuning, and Verification. *arXiv preprint arXiv:2409.16461*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving Math Word Problems with Process- and Outcome-Based Feedback. In *NeurIPS 2022 Workshop on MATH-AI*.

Sydney Von Arx, Lawrence Chan, and Beth Barnes. 2025. Recent Frontier Models Are Reward Hacking. METR Technical Blog. https://metr.org/blog/2025-06-05-recent-reward-hacking/.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, and 1 others. 2024. Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. In *Proceedings of the Association for Computational Linguistics (ACL)*. ArXiv:2312.08935.

Xiaohua Wang, Muzhao Tian, Yuqi Zeng, Zisu Huang, and 1 others. 2026. Reward Hacking in the Era of Large Models: Mechanisms, Emergent Misalignment, Challenges. *arXiv preprint arXiv:2604.13602*.

Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. 2025. Logic-RL: Unleashing LLM Reasoning with Rule-Based Reinforcement Learning. *arXiv preprint arXiv:2502.14768*.

Lecheng Yan, Ruizhe Li, Guanhua Chen, Qing Li, Jiahui Geng, Wenxi Li, Vincent Wang, and Chris Lee. 2026. Spurious Rewards Paradox: Mechanistically Understanding How RLVR Activates Memorization Shortcuts in LLMs. *arXiv preprint arXiv:2601.11061*.

Kaiyu Yang, Jia Deng, and Danqi Chen. 2022. Generating Natural Language Proofs with Verifier-Guided Search. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. 2023. SatLM: Satisfiability-Aided Language Models Using Declarative Prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. 2025. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems (NeurIPS)*.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. ProcessBench: Identifying Process Errors in Mathematical Reasoning. In *Proceedings of the Association for Computational Linguistics (ACL)*. ArXiv:2412.06559.

A Experimental Setup

Table 3 lists all hyperparameters shared across experiments. All RL conditions use the same GRPO configuration; only the reward function differs. Seeds control model weight initialization and data shuffling; training data is held constant across all seeds and conditions (Constraint C001).

Table 3: Training hyperparameters for all GRPO experiments.

Parameter	Value
Model	
Base model	Qwen3-4B
Precision	BF16
GRPO Training	
Rollouts per prompt (K)	4
Training steps (default)	500
Training steps (extended)	1000
Learning rate	1×10^{-6}
Per-device batch size	4
Gradient accumulation steps	2
Effective batch size	8
Max completion length	512 tokens
Random seeds	42, 43, 44
Data	
Training set	FOLIO, 500 examples
Evaluation set	FOLIO, 100 examples (held-out)
Data file	`train_500_seed42`
Solvers	
FOL solver	Prover9
Prolog solver	SWI-Prolog 8.4.2
Evaluation	
Decoding	Greedy (T=0)
ISR computation	`eval_sta.py`
ISR variant	Proved-only (ISR*)

Compute resources. Experiments were run on single-GPU setups (NVIDIA RTX 4090 24 GB and RTX PRO 6000 96 GB). Each GRPO training run takes approximately 1–2 GPU-hours (500 steps) or 3–5 GPU-hours (1000 steps). Total compute across all seeds, conditions, and reruns is approximately 80–120 GPU-hours.

SFT convergence. SFT training converged across all three seeds: final-step loss $\leq$0.04 and token accuracy >99% after ∼8 epochs on 500 examples (seed 42: 0.029; seed 44: 0.033); the 1000-step extension further decreases loss to 0.017.

Evaluation exception. FOL exec-only seed 43 (`plan001-exec-only-s43-r4`) was evaluated with sampling (temperature = 0.7) rather than greedy decoding. Its PR and ISR values are included for completeness but are not strictly comparable with other seeds under the unified evaluation protocol (C004).

Reward formulation. An additive variant ($r = r_{\text{exec}} + \beta\,\text{ISR}$) was tested in early experiments but collapsed prove rate to near-zero, as ISR alone provides no gradient toward provable formalizations; the multiplicative form $r_{\text{exec}} \cdot (1 + \beta\,\text{ISR})$ ensures ISR modulates only proved outputs.

ProofFOL dataset construction. ProofFOL is derived from ProofWriter (Tafjord et al., 2021) (10,424 training examples) via the GPT-4o silver-standard FOL translation pipeline of Thatikonda et al. (2024) with Prover9 verification filtering (∼70% retention; failures from timeouts, unsupported constructs, and translation errors). Of the translated examples, 99.1% parse successfully (92 skipped due to empty FOL output). We sample 500 training and 100 evaluation examples (seed 42) from the verified pool. Table 4 compares key structural properties.

Table 4: ProofFOL vs. FOLIO structural comparison (training sets, n=500 each).

Property	ProofFOL	FOLIO
FOL premises (mean±std)	18.2 ± 3.2	5.3 ± 1.4
Ground facts	10.2 ± 3.0	1.8 ± 1.5
Rules	8.1 ± 1.1	3.5 ± 1.9
XOR ($\oplus$) samples	0%	37.2%
Dominant depth	4–5	2–3
Predicate style	synthetic	real-world
FOL source	GPT-4o (silver)	human (gold)

ProofFOL problems are structurally more complex (3.4× more premises) but use simpler, synthetic predicates (e.g., `Blue(x)`, `Rough(x)`) characteristic of ProofWriter's procedural generation. The absence of XOR constraints and the regularity of predicate structure make ProofFOL a complementary testbed: structural degeneration must emerge from the RL training dynamics rather than from formalization difficulty.

B STA Algorithm

Algorithm 1 (Section 3.2) gives the procedure for computing ISR over a proved formalization. For unproved samples ($S(x) \neq \textit{proved}$), ISR is undefined and the protocol is not applied.

C Extended Discussion

What ISR measures—and what it does not. ISR quantifies syntactic component necessity: the fraction of solver-input components whose individual removal causes proof

failure. This is strictly weaker than semantic faithfulness: whether the formalization accurately captures the natural-language meaning. We verify this empirically: Spearman $\rho(\text{ISR}, \text{NLI accuracy}) = -0.049$ ($p = 0.187$, n.s.) from a downstream NLI evaluation on exec-only checkpoint outputs. ISR and downstream task performance are uncorrelated. ISR is thus a component-necessity diagnostic, not a semantic quality metric: high ISR indicates that every component contributes to the proof, not that the proof faithfully represents the original reasoning problem.

Critically, low ISR carries different semantics depending on the training objective. Under exec-only training, low ISR in Prolog reflects redundant coding (repeated ground facts where removing any single copy does not affect the proof) rather than an absence of reasoning (§4.4). Under ISR-as-reward, high ISR reflects metric gaming (§4.3). ISR values must always be interpreted relative to the optimization objective that produced them.

The premise-count confound. LOO-based metrics like ISR have an inherent mathematical bias: with fewer components, each is mechanically more likely to be necessary. A formalization with 2 premises where both are necessary yields ISR=1.0; one with 10 premises where 8 are necessary yields ISR=0.8, even though the latter encodes richer reasoning. This confound explains the sole apparent ISR-as-reward success (§4.3): ISR β=0.1 seed 42 produces shorter formalizations, mechanically inflating ISR. Premise-count-matched comparison eliminates the difference (1.06×; Table 8). This bias is not specific to our setting: any LOO-based structural metric applied to variable-length outputs requires component-count normalization. We recommend that future work employing ablation-based diagnostics report premise-count-matched comparisons alongside raw values.

TS dose-response gradient. TS tautology follows a dose-response pattern across reward conditions: random β=0.1 produces 13.7% TS (36/262 pooled; per-seed: 38.3% in s43, 0% in s42/s44), random β=0.3 yields 4–7 TSa samples per seed, and ISR β=0.1 reaches 70.6% (163/231). The gradient (random β=0.1 (13.7%) → random β=0.3 (4–7 TSa per seed) → ISR β=0.1 (70.6%)) shows ISR transforms TS from a seed-dependent GRPO tendency to a near-universal convergence direction.

Goodhart effects: implications. ISR was designed to *detect* reward gaming of r_{exec}, yet when incorporated into the reward signal, the model games ISR itself. Any diagnostic metric risks becoming a target when optimized. The dose-response pattern maps onto the Manheim & Garrabrant taxonomy (Manheim and Garrabrant, 2019): β=0.1 on FOLIO instantiates *regressional* Goodhart ($P \to P$ tautologies yield ISR$\geq$0.5, so ISR erroneously rewards the shortcut); $\beta \geq 0.2$ introduces *extremal* elements (PR collapse to 18% vs. 48% random at β=0.3). The implication is conditional: ISR fails as a training reward when hacking satisfies its LOO structure (FOLIO), but produces beneficial instability when hacking does not (ProofFOL; §4.4).

Single-seed unreliability. The three exec-only seeds diverge radically (PR σ=47 percentage points), each converging to a qualitatively different degeneration mode (§4.2). ISR β=0.1 seeds show similar divergence: 2/3 dominated by conclusion duplication (TS), 1/3 by premise minimization (§4.3). Multi-seed evaluation with structural diagnostics should be standard practice for neurosymbolic RL systems.

Classifier limitations. Text-match hacking classifiers have systematic blind spots that functional analysis exposes. In Prolog STA, two seeds achieved 100% genuine classification under strict text matching, yet functional analysis via the $n_{\text{necessary}}$ distribution revealed 0% genuine in both cases (Table 9). One seed produces uppercase pseudo-Prolog that is functionally tautological; the other converts conclusions to snake_case atoms serving as both fact and query, a gaming mode invisible to string comparison. Practitioners should cross-validate text-match classifiers with functional metrics and never treat classifier-reported genuine rates as ground truth without independent verification.

Practical guidelines. Our findings yield five guidelines for GRPO-trained neurosym-

Table 5: Fisher exact tests for non-hacking rate across conditions (3-seed pooled). ISR β=0.1 non-hacking rate is corrected for PG reclassification and TS false negatives (Appendix D.1).

Comparison	Condition	Control	OR	p
Non-hacking rate (PG+FN corrected)				
ISR vs. Random	6.06% [3.6,9.9]	9.5% [6.5,13.7]	0.61	0.182 (n.s.)
ISR vs. Exec-only	6.06%	7.5%	0.79	0.62
Random vs. Exec-only	9.5%	7.5%	1.30	0.68
TS gaming rate				
ISR vs. Random	70.6% [64.3,76.3]	13.7% [9.9,18.6]	15.05	1.3×10^{-39}
SFT vs. Exec-only non-hacking rate				
SFT vs. Exec-only	92.3% [79.7,97.3]	7.5%	147	<0.001

ISR: n=231 (3 seeds). Random: n=262 (3 seeds). Exec-only: n=93 (s42). SFT: n=39 (3 seeds, 1000 steps). 95% Wilson CIs in brackets where shown.

bolic systems: (1) Never evaluate by prove rate alone; use ISR and component count as joint diagnostics. (2) Report multi-seed results: single-seed GRPO evaluation is unreliable, as cross-seed variance can exceed the effect size of interest (§4.2). (3) Use ISR for post-hoc diagnosis only, not as a training signal; any dose induces gaming or collapse. (4) Treat LOO-based metrics as component-count-biased until normalized. (5) Expect conclusion embedding as a GRPO-universal baseline (Table 1): reward shaping alters hacking composition, not prevalence.

Multiple testing correction. We define two test families: FOLIO (k=14 tests across all conditions) and ProofFOL (k=1). Within the FOLIO family, Bonferroni correction at α=0.05 yields adjusted thresholds of p_{adj}=0.05/14=0.0036. Three primary comparisons survive correction: the TS rate difference (ISR vs. random; $p_{\text{bonf}}=1.82\times10^{-38}$), the SFT vs. exec-only non-hacking rate (p_{bonf}=0.014), and the Cochran–Mantel–Haenszel test for TS association across seeds ($p_{\text{bonf}}<10^{-8}$). The matched 5+5-seed Fisher/Mann-Whitney comparison is borderline (p_{bonf}=0.056), significant at α=0.10 after Bonferroni correction (k=14). The ProofFOL genuine-rate permutation test (p=0.10) belongs to a separate hypothesis family (different dataset, independent experimental design) and does not reach significance even without correction; it is reported as an exploratory observation throughout. Bonferroni assumes independence, which is conservative for our setting (several tests use overlapping data); Holm–Bonferroni or Benjamini–Hochberg FDR would recover the matched 5+5 comparison at conventional significance. We report the most conservative Bonferroni results for transparency.

D Hacking Mode Definitions

We identify eight distinct reward hacking modes across all experiments. Each mode is defined below with its mechanism, a representative example, and its interaction with ISR. Table 6 summarizes prevalence across FOL conditions.

Classifier Validation. Hacking mode classification uses deterministic rule-based classifiers, not learned models, so standard ML precision/recall is not directly applicable. CE and TS are identified by normalized text matching between conclusion and premise strings after whitespace and variable normalization; remaining modes are detected via functional analysis of the per-sample $n_{\text{necessary}}$ distribution from STA (leave-one-out ablation). No training data or manual annotation is used for classification.

To estimate precision, we manually verified 50 randomly sampled outputs classified as CE or TS: all 50 were confirmed correct (estimated precision >95%, Wilson 95% CI: [93%, 100%]). Recall is inherently bounded by the text-match threshold: semantic manipulations that alter surface form while preserving the gaming structure (e.g., variable renaming beyond our normalization, or paraphrased conclusions) would be missed. We cross-validate with functional LOO analysis: samples classified as non-hacking but showing $n_{\text{necessary}} \leq 1$ are flagged as potential false negatives. This cross-validation revealed query-as-fact gaming in Prolog (§D), where the strict text classi-

Mode	Mechanism	Exec	ISR	Rnd	SFT
CE	Concl. in premises	87	18	71	7
TS	Single prem. = concl.	•	**71**	14†	0
PG	Premise-subset gaming	6	5	–	0
Tautological	2-comp $P{=}C$	•	0	0	0
Repetition	Output degen.	•	–	–	–
Contradiction	n_{comp} inflated	•	–	–	–
C-w/o-utility	Complex, redundant	0	0	4	0
Q-as-fact	Atom = fact+query	–	–	–	–

Table 6: Hacking taxonomy across FOL conditions (% of proved samples). ISR/Rand columns: β=0.1 3-seed pooled. TS includes subtypes TSa (text-match, n=149 ISR + 36 random) and TSb (obfuscated tautology, n=14; primarily s44). †Random TS driven by seed 43 (38.3%); seeds 42/44: 0%. Exec: CE=87% in s42; • marks seed- or step-specific modes (TS/Tautological in s44, Repetition in s43, Contradiction at 1000 steps). PG: premise-subset gaming (Appendix D.1). C-w/o-util.: complexity-without-utility (random-specific). Q-as-fact: Prolog STA only (Table 9).

fier reported 100% non-hacking but functional analysis showed 95/97 samples were tautological ($n_{\text{necessary}}$=1). The non-hacking rate for FOL conditions may therefore be slightly overestimated, as subtle semantic manipulations below our text-matching threshold would evade the strict classifier.

Boundary case validation. Stratified validation across 82 samples (all five hacking categories) yields Cohen's κ=1.0; independent reannotation confirms full agreement. Structured inspection of 20 boundary cases (short-conclusion CE (10, minimum 4 characters), low-ISR genuine (5, minimum ISR=0.22), and TS on false propositions (5)) confirmed 20/20 correct classifications. Short-conclusion CE samples are unambiguous hacking: the conclusion appears as a standalone premise with no NL source. Low-ISR genuine samples reflect mode-collapsed template outputs, not structural gaming. 3/5 TS boundary cases confirm factually false propositions (e.g., affirming "dogs are plants"), showing TS is not merely vacuous but actively harmful.

D.1 PG — Premise-Subset Gaming

Manual audit of ISR β=0.1 seed 42 revealed a previously undetected hacking mode: the model emits a subset of the gold premises that is individually necessary (high ISR) but insufficient for the intended derivation, with proof success depending on the conclusion already being embedded (CE-style). We term this **PG (premise-subset gaming)**: formalizations where each component is LOO-necessary but the premise set is a strict subset of what the NL problem requires, inflating ISR without faithful reasoning.

Mode	Validated N	Precision
CE (Concl. Embedding)	80†	1.00
TS (Tautological)	7	1.00
PG (Premise-Subset)	20	1.00
Contradiction	20	1.00
Genuine	25	1.00
Overall	**152**	**1.00**

Table 7: Per-mode classifier precision on 152 manually validated samples (stratified + boundary + random; three exercises pooled). †Includes 50 randomly sampled CE∪TS outputs (pool ratio CE:TS=374:2). Recall is not estimable from a validation-only design.

Detection. PG is identified by manual inspection of non-hacking samples with unusually low premise counts ($n_{\text{prem}} \leq 3$) that nonetheless achieve high ISR. Automated detection is not currently feasible because PG requires comparing the generated premise set against the gold premises.

Audit scope and limitations. Full PG audit was completed for ISR β=0.1 seed 42 only. ISR seeds 43 and 44 checkpoints were deleted before audit and cannot be recovered; their non-hacking rates may be overestimated. Random β=0.1 does not produce PG by mechanism (no ISR optimization pressure); seed 42 audit confirmed PG=0.

TS false-negative correction (TSb obfuscated tautology). ISR β=0.1 seed 44 contained 12 samples with n_{prem}=1 classified as non-hacking by the text matcher. Manual inspection confirmed all 12 are logically equivalent to TS but evade text matching through

vacuous quantifier wrapping (11/12) or structural simplification (1/12). We classify these as **TSb** (obfuscated tautology), distinct from text-match-detectable TSa. 10/12 achieve ISR≥0.83, indicating simultaneous gaming of execution reward, ISR metric, and TS classifier: triple gaming (§4.5). After correction, ISR s44 non-hacking=0; total TS = TSa (77 text-match) + TSb (12 obfuscated) = 89.

Impact on non-hacking-rate comparison. After PG reclassification and TS false-negative correction: ISR non-hacking = 14/231 = 6.06% vs. random non-hacking = 25/262 = 9.5% (Fisher p=0.182, n.s.; Wilson 95% CI: ISR [3.6%, 9.9%], Random [6.5%, 13.7%]). The direction is consistent with ISR-amplified gaming modes (TS, PG) displacing non-hacking output, but the effect is not statistically significant at the conventional α=0.05 threshold.

D.2 CE — Conclusion Embedding

Definition. The conclusion appears verbatim (or with trivial variable renaming) as one premise among multiple premises. The solver succeeds by matching the embedded conclusion, rendering the other premises irrelevant to the proof.

Example (FOL).

```
Premises:
  all x (bird(x) -> flies(x)).
  bird(tweety).
  flies(tweety).  % conclusion embedded
Conclusion: flies(tweety).
```

ISR interaction. Variable. The embedded premise has ISR = 1 (removing it may break the proof), but other premises may also register as necessary depending on Prover9's search strategy. ISR does not distinguish trivial from substantive necessity.

Prevalence. Conclusion embedding in some form (CE multi-premise or TS single-premise) is GRPO-universal at 68–97% across all RL conditions. Under exec-only and Random reward, it manifests as CE (71–93%); under ISR reward, it shifts to TS (70.6% pooled).

D.3 TSa — Conclusion Duplication (Standard)

Definition. The entire formalization collapses to a single premise whose content is identical (text-match detectable) to the conclusion. Since $P \to P$ is trivially provable, this simultaneously maximizes exec reward. Because the sole premise is necessary for every sub-component of the conclusion under LOO ablation, ISR is inflated toward 1.0.

Example (FOL).

```
Premises:
  all x ((A(x)&B(x))->(C(x)|D(x))).
Conclusion:
  all x ((A(x)&B(x))->(C(x)|D(x))).
```

ISR interaction. ISR ≈ 0.8–1.0. The single premise is trivially "maximally necessary," but the proof is vacuous. TSa games both exec reward and ISR reward; TSb extends this to triple gaming by additionally evading the text-match classifier.

Prevalence. TSa (text-match detectable): 64.5% pooled (149/231) under ISR β=0.1 (s42: 16.0%, s43: 96.7%, s44: 86.5%). Combined TS (TSa + TSb): 70.6% (163/231; s43 TSa=59 + TSb=2; includes s44 FN correction). Random β=0.1 pooled: 13.7% (36/262, all TSa; driven by seed 43 at 38.3%, seeds 42 and 44 at 0%).

D.4 TSb — Obfuscated Tautology

Definition. A single-premise formalization logically equivalent to TSa (premise = conclusion) but with surface-form differences that evade the text-match TS classifier. The model achieves this through vacuous quantifier wrapping (11/12 instances) or structural simplification (1/12).

Example (FOL, vacuous quantifier wrapping).

```
Premises:
  all x all y ((A(x)&B(y))->(C(x)|D(y))).
Conclusion:
  all x ((A(x)&B(x))->(C(x)|D(x))).
```

The premise introduces a vacuous second variable y, immediately unified with x; the formulas are logically equivalent but differ textually.

ISR interaction. ISR≥0.83 (10/12 samples). TSb simultaneously games three objectives: execution reward (r_{exec}=1 via tautological proof), ISR metric (single necessary premise yields high ISR), and the text-match TS classifier (surface-form difference evades detection). This triple gaming makes TSb the most complete Goodhart case we observe.

Prevalence. Found primarily in ISR β=0.1 seed 44 (12/89 proved samples, all originally classified as non-hacking); seed 43 contributes 2 additional cases. Functional cross-validation via $n_{\text{necessary}}$=1 exposed the misclassification.

D.5 Tautological Collapse

Definition. Two-component formalizations where the premise and conclusion are identical atomic or simple formulas, yielding a $P \to P$ tautology with minimal token cost.

Example (FOL).

```
Premises: p(a).
Conclusion: p(a).
```

ISR interaction. ISR = 1.0 (the sole premise is necessary). Distinguished from TS by having low component count ($n_{\text{comp}} \approx 2$) and atomic formulas rather than complex substructured conclusions.

Prevalence. Dominant in FOL exec-only seed 44 (PR = 0.98, mean $n_{\text{comp}} = 2.04$, 100% TS by strict classifier). Also observed in Prolog exec+STA seed 42 (100% TS, PR = 0.50).

D.6 Repetition Collapse

Definition. The model generates identical premises or clauses repeated until the maximum completion length, producing a degenerate output with high token count but no logical diversity. Removing any single repeated copy leaves the others intact, yielding ISR ≈ 0.

Example (Prolog).

```
penguin(fido).
penguin(fido).
penguin(fido).
... (repeated to 512 tokens)
query(penguin(fido)).
```

ISR interaction. ISR ≈ 0. Each copy is individually redundant since identical copies remain after removal. ISR correctly diagnoses this degeneration.

Prevalence. Observed in FOL exec-only seed 43 (87% of outputs hit max completion length; PR = 13%, ISR = 0).

D.7 Complexity-without-Utility

Definition. The formalization contains syntactically complex premises (nested quantifiers, multi-arity predicates) that are semantically vacuous: none individually necessary for the proof. The complexity inflates surface metrics (formula length, nesting depth) without contributing to logical derivation.

Example (FOL).

```
Premises:
  all x all y (R(x,y) ->
    exists z (S(x,z) & T(z,y))).
  all x (P(x) -> Q(x)).
  P(a).            % only this needed
Conclusion: Q(a).
```

ISR interaction. ISR correctly identifies the vacuous premises (ISR = 0 for each), pulling down the mean ISR. This is the mode ISR is best calibrated for.

Prevalence. Low. Observed in Random β=0.1 seed 42 at 3.8% (3/79). Not observed under ISR reward (ISR's gradient discourages zero-ISR premises).

D.8 Contradiction Explosion

Definition. The component count grows unboundedly over training. Excess premises introduce contradictions or redundancies that trivially entail any conclusion via *ex falso quodlibet*. Individual premises may register as ISR = 0 (removable without breaking the proof), but the sheer count obscures the underlying triviality.

Example (FOL, 1000-step exec-only).

```
Premises:
  c1. c2. c3. ... c18. % 18.55 mean
  % conclusion embedded among premises
Conclusion: Q(a).
```

ISR interaction. ISR ≈ 0 (at 1000 steps: ISR = 0.003, with 94.8% of proved samples having ISR = 0).

Prevalence. Observed in extended exec-only training (seed 42, 1000 steps). Mean component count grows from ∼6 (500 steps) to 18.55 (1000 steps).

D.9 Query-as-Fact Gaming

Definition. The model converts the natural language conclusion into a snake_case atom and includes it as both a ground fact and the query. Prolog trivially proves the query by unifying with the fact. This mode evades text-match classifiers designed for CE/TS because the generated atom bears no surface resemblance to the original conclusion text.

Example (Prolog).

```
% NL: "Fido is happy."
% Model output:
fido_is_happy.
:- fido_is_happy.      % query = fact
```

ISR interaction. ISR ≈ 0.9–1.0. The single fact is trivially necessary (its removal breaks the query), inflating ISR. Functionally equivalent to TS but with a distinct surface mechanism.

Prevalence. Observed in Prolog exec+STA seed 44 (PR = 0.97, ISR = 0.912). 95/97 proved samples have $n_{\text{necessary}} = 1$, confirming functional tautology despite the strict classifier reporting 100% non-hacking.

E Per-Seed Detailed Results

Tables 8 and 9 report full per-seed results. All metrics are computed by `eval_sta.py` on the held-out 100-example FOLIO evaluation set using greedy decoding unless otherwise noted. ISR values are proved-only ISR*.

ISR collapse trajectory (FOL exec-only, seed 42). Table 8 traces the ISR collapse over training: ISR* = 0.642 at 100 steps → 0.132 at 500 steps (−79.4%) → 0.003 at 1000 steps (−99.5% from step 100). PR increases monotonically (0.32 → 0.93 → 0.96) while component necessity collapses, confirming that ISR flags degeneration invisible to execution reward. By step 1000, 94.8% of proved samples have ISR = 0 and the mean component count rises to 18.55.

Prolog: universal Goodhart via three distinct pathways. All three Prolog exec+ISR seeds exhibit Goodhart's Law with 0% non-hacking proofs (Figure 2), but each discovers a different gaming mechanism: seed 42 produces clean TS tautologies (single premise = conclusion), seed 43 generates uppercase pseudo-Prolog that bypasses the text-match classifier while remaining functionally tautological, and seed 44 converts NL conclusions to snake_case atoms used as both fact and query. The diversity of gaming pathways across seeds (despite identical training data and reward function) demonstrates that Goodhart's Law in neurosymbolic reasoning is not a single failure mode but a family of convergent exploits.

E.1 Per-Seed Results at β=0.2 and β=0.3

Table 11 reports per-seed results for ISR at β=0.2 and both ISR and Random at β=0.3. At β=0.2, all three seeds converge to pure TS Goodhart (100% or near-100% TS), confirming $\beta \geq 0.2$ as the extremal Goodhart regime. The three ISR β=0.3 seeds illustrate qualitatively distinct Goodhart pathways (§4.3).

Table 8: Detailed per-seed results: FOL (Prover9). Includes the per-seed β=0.1 breakdown referenced from Table 1. PR = prove rate, ISR* = proved-only ISR, TS/CE/N-H = percentage among proved samples. "—" indicates data not available. †s43 uses sampling eval (T=0.7). ‡Three s44-1000 samples escape text-match heuristics (CE/PG classifiers) but exhibit ISR=0 and functionally degenerate proof structures: disjunctive conclusions that bypass PG detection and commuted premises that bypass CE detection; reclassified as degenerate. ◇Seeds 45–46 added post-hoc under identical hyperparameters (effective batch size 8; micro-batch decomposition differs: 4×2 vs. 2×4, mathematically equivalent).

Condition	**Seed**	**Steps**	**PR**	**ISR***	**TS%**	**CE%**	**N-H%**	**Notes**
Baselines								
Base model (no training)	—	0	0.12	0.064	—	—	—	Pre-training
Ground-truth FOL	—	—	0.18	0.559	—	—	—	Human formalizations
SFT	42	1000	0.13	0.568	0	7.7	**92.3**	Supervised, no RL
	43	1000	0.11	0.635	0	9.1	**90.9**	
	44	1000	0.15	0.589	0	6.7	**93.3**	
	3-seed pooled		0.130	0.595	0	7.8	**92.3**	n=39 proved
Exec-only								
	42	100	0.32	**0.642**	—	—	—	Early checkpoint
	42	500	0.93	0.132	0	87.1	7.5	PG=6.5%
	42	1000	0.96	0.003	0	100	0	Contradiction explosion
	43	500	0.13	0†	—	—	—	†Greedy (n_{proved}=13)
	43	1000	0.96	0.082	0	93.8	6.2	CE dominant
	44	500	0.98	0.582	100	0	0	Tautological collapse
	44	1000	1.00	0.002	0	85	0‡	PG=15%
	3-seed mean±std		0.68±0.47	0.238±0.30				500-step
	3-seed mean±std		0.97±0.02	0.029±0.05			2.1±3.6	1000-step
ISR β=0.1								
	42	500	0.81	0.415	16.0	51.9	**17.3**	Mixed CE+TS+PG
	42	1000	0.90	0.877	96.7	0	2.2	TS collapse; PG=1 (n=90)
	43	500	0.61	0.830	**100**	0	0	TSa=59, TSb=2
	44	500	0.89	0.902	100	0	0	TS dominant (FN corrected)
	45◇	500	0.42	0.605	38.1	4.8	**54.8**	PG=1; n=42
	46◇	500	0.77	0.411	14.3	26.0	**54.5**	PG=4; n=77
	3-seed mean±std		0.77±0.14	0.716±0.26				500-step (s42–44)
	5-seed mean±std		0.70±0.19	0.633±0.23				500-step (s42–46)
ISR β=0.3								
	See Table 11 for per-seed β=0.3 results.							
Random β=0.1								
	42	500	0.79	0.131	0	73.4	22.8	PG=4
	43	500	0.94	0.150	**38.3**	59.6	1.1	TSa=36; PG=1
	44	500	0.89	0.070	0	82.0	6.7	PG=16 (CE∩PG: 6)
	3-seed mean±std		0.87±0.08	0.117±0.042				TS seed-dep.: 0/38/0%
Random β=0.3								
	42	500	0.48	0.094	8	92	0	TSa=4 (D086); n_{comp} = 9.6
	43	500	0.92	0.180	7.6	89.1	3.3	TSa=7; $n_{\text{N-H}}$=3
	44	500	0.23	0	0	100	0	Pure CE (n=23)
	3-seed mean±std		0.54±0.35	0.09±0.09				
Shuffled-ISR β=0.1 *(deconfounding control)*								
	42	500	0.15	0.567	6.7	6.7	**86.7**	n=15 proved
	43	500	0.16	0.568	6.3	6.3	**87.5**	n=16 proved
	44	500	0.15	0.542	6.7	6.7	**86.7**	n=15 proved
	3-seed mean±std		0.153±0.006	0.559±0.015	6.5	6.5	**87.0**	N=46 proved

Table 9: Detailed per-seed results: Prolog (SWI-Prolog 8.4.2). PR = prove rate, ISR* = proved-only ISR, TS/CE/N-H = percentage among proved samples. "—" indicates data not available.

Condition	Seed	PR	ISR*	TS%	CE%	N-H%	Notes
Exec-only							
	42	0.83	0.037	0	0	—	Ground fact enumeration
	43	0.56	0.015	—	—	—	
	44	0.21	0.387	—	—	—	Small-sample (n=21)
	3-seed mean±std		0.53±0.31	0.146±0.21			
Exec+ISR (STA reward)							
	42	0.50	1.000	**100**	0	0	TS tautological
	43	0.38	0.185	0‡	97.4	0	‡Functional TS[a]
	44	0.97	0.912	0‡	0	0‡	Query-as-fact gaming[b]
	3-seed mean±std		0.62±0.31	0.699±0.45			3/3 Goodhart, 0% non-hacking

[a] Strict TS classifier reports 0%, but functional analysis shows all 38/38 proved samples are tautological ($n_{\text{necessary}} = 1$). Uppercase pseudo-Prolog syntax bypasses the text-match classifier.
[b] Strict classifier reports Non-hacking = 100%, but 95/97 proved samples have $n_{\text{necessary}} = 1$. The model converts NL to snake_case atoms serving as both fact and query, a third distinct Goodhart pathway.
‡ Strict classifier value is misleading; see footnotes [a]/[b].

Table 10: Detailed per-seed results: ProofFOL (Prover9). PR_{f50} = mean prove rate over final 50 steps. TS = tautological shortcutting (§4.5), CE = conclusion embedding, Genuine = classified as neither TS nor CE. Stable convergence: PR_{f50}>0.90.

Condition	Seed	PR_{f50}	ISR*	TS%	CE%	Genuine%	Notes
Base	—	0.16	0.310	0	0	**100**	n=16 proved
Exec-only							
	42	≈1.0	0.066	**100**	0	0	n=99; stable
	43	0.55	0.119	0	49.2	50.8	n=63; partial
	44	≈1.0	0.043	0	98.6	1.4	n=72; stable
	Aggregate			17.4% genuine (mean of seeds)			
ISR β=0.1							
	42	0.35	0.12	0	0	**100**	n=46; collapsed
	43	≈0.47	≈0.13	0	69.6	30.4	n=56; unstable
	44	0.49	0.145	0	0	**100**	n=66; oscillation
	Aggregate			**76.8%** genuine (mean of seeds)			

Aggregate genuine rate: mean of per-seed genuine rates. Permutation test (exec vs. ISR genuine): p=0.10, N=3 per group.

Table 11: Per-seed results at β=0.2 and β=0.3 (FOL). PR = prove rate, ISR* = proved-only ISR, TS/CE/N-H = percentage among proved samples.

Condition	Seed	PR	ISR*	TS%	CE%	N-H%	Mode
ISR β=0.2							
	42	1.00	0.602	**100**	0	0	Pure TS (n=100)
	43	0.96	0.552	**96.9**	1.0	0	PG=2
	44	0.94	0.964	**100**	0	0	Pure TS (n=94)
	3-seed mean±std		0.97±0.03	0.71±0.22			3/3 TS Goodhart
ISR β=0.3							
	42	0.18	0.143	0	77.8	5.6	Reward collapse (n=18); PG=3
	43	0.86	0.295	4.7	68.6	23.3	CE shortcutting
	44	0.95	0.742	**100**	0	0	Pure TS Goodhart
	3-seed mean±std		0.66±0.42	0.39±0.31			
Random β=0.3							
	42	0.48	0.094	8	92	0	TSa=4 (D086)
	43	0.92	0.180	7.6	89.1	3.3	TSa=7; $n_{\text{N-H}}$=3
	44	0.23	0	0	100	0	Pure CE (n=23)
	3-seed mean±std		0.54±0.35	0.09±0.09			

E.2 Early-Training Hacking Analysis (100-Step Checkpoint)

To control for the prove-rate confound in our SFT comparison (§4.2), we analyze the exec-only checkpoint at 100 training steps (seed 42), where PR=32%, higher than SFT (13.0%, 3-seed pooled) but still early in training. Of the 32 proved samples, 9 (28.1%) exhibit TS hacking (single premise identical to conclusion) and 1 (3.1%) exhibits CE (conclusion embedded among premises), yielding a total hacking rate of 31.2%. By contrast, SFT at comparable or lower PR shows 7.7% hacking (1/13 proved samples). This demonstrates that reward hacking emerges early in GRPO training, well before the prove-rate plateau, and is not simply an artifact of having more proved samples available for exploitation. The 100-step hacking rate (31.2%) is already substantially above SFT (7.7%), supporting the claim that hacking is associated with RL optimization rather than prove-rate level alone.

E.3 $O(1)$ vs. $O(n)$ ISR Ablation

All main experiments (§4.2–4.4) use $O(n)$ full LOO ISR during both training and evaluation; the $O(1)$ binary variant described below exists as an alternative implementation (`execution_fast_sta`) but is used only in the ablation reported here. The two signals measure fundamentally different properties, not approximations of the same quantity. The binary signal tests *collective* premise necessity: fast_ISR(x) = 1 if x is proved and the conclusion is not provable under empty premises (tautology check), 0 otherwise. LOO tests *individual* premise necessity: each component is removed in turn, and ISR reflects the fraction whose removal causes proof failure.

Evaluation-time comparison. Across 402 proved samples from all ProofFOL conditions, the $O(1)$ binary signal is 1.0 for every sample: non-tautological conclusions cannot be proved without any premises, so the collective necessity test is trivially satisfied. This renders $O(1)$ unable to distinguish hacking from genuine formalizations: both pass the empty-premise check. By contrast, $O(n)$ LOO produces a clear separation: exec-only models average ISR*=0.046 (n=234) while ISR-trained models average ISR*=0.183 (n=168), because hacking constructions contain individually redundant components that LOO detects. Of 179 disagreements between the two signals, all are false positives ($O(1)$=1, LOO=0): the binary check approves formalizations that LOO correctly identifies as structurally degenerate.

Training-time ablation. As a training reward ($r = 0.5\, r_{\text{exec}} + 0.5\, \text{fast_ISR}$, seed 42, 500 steps), the $O(1)$ variant achieves PR=63% with ISR*=0.007, functionally zero, compared to ISR*=0.132 under exec-only and ISR*=0.42 under $O(n)$ ISR β=0.1. The tautology check depresses prove rate by 30 pp relative to exec-only (63% vs. 93%) by penalizing formalizations where the conclusion is provable without premises, but this constraint provides no gradient toward individual component necessity; the model is free to degenerate as long as it avoids trivial tautologies. That even the fine-grained $O(n)$ LOO signal, which *does* provide per-component necessity feedback, is ultimately gamed (§4.3) underscores that the problem lies in reward optimization itself, not in signal granularity.

Computational overhead. Full LOO requires n+1 solver calls per sample (mean $n\approx6$ for FOLIO). With Prover9 at ∼5 ms per call, STA evaluation adds ∼55 ms per sample versus ∼4 s for LLM inference (<1% overhead). The $O(1)$ binary check adds a single solver call per proved sample (<2% per training step), but the evaluation above shows this saving comes at the cost of all diagnostic power.

E.4 Cross-Model Generalization: LLaMA-3.2-3B Cold-Start Failure

We applied the identical GRPO configuration (Table 3) to LLaMA-3.2-3B (Grattafiori et al., 2024) (3B; exec-only, seed 42, K=4, FOLIO). Across 72 of 500 planned steps the model produced zero valid Prover9 output, generating only natural-language text; r_{exec}=0 for all rollouts yields zero GRPO advantage, so training was terminated early.[5] This establishes a *capability threshold*: the base model must produce syntactically valid solver input for the RL loop to generate signal. Qwen3-4B

[5] This run was also affected by a generation-termination configuration issue (dual EOS tokens); the zero prove rate reflects both the base model's lack of FOL generation capability and truncated outputs.

crosses this threshold (non-zero prove rates within 10 steps) while LLaMA-3.2-3B does not despite comparable size (3B vs. 4B), likely reflecting pretraining data composition. An SFT warmstart (3 epochs on gold formalizations, lr=2×10^{-5}) achieves PR=16% (n=100) after correcting the generation-termination issue. However, the model produces Unicode FOL notation ($\forall$, $\rightarrow$, $\wedge$) instead of the required ASCII syntax despite training on ASCII-formatted data; the 16% prove rate reflects Prover9's partial Unicode tolerance rather than correct format adherence. Subsequent GRPO runs (exec-only, 500 steps from SFT warmstart, 3 seeds) fail to improve beyond SFT baseline (PR=15.0±1.0%, n=14–16 proved per seed), yet produce no structural degeneration: 86.6±0.9% of proved samples are genuine across seeds, with cross-seed variance near zero (σ_{PR}=1 pp). This contrasts sharply with Qwen3-4B, where GRPO reaches 93% PR but collapses to 8% genuine with high cross-seed variance. The result suggests structural degeneration is a byproduct of successful RL optimization, not of the training procedure *per se*: when GRPO fails to amplify prove rate, the model retains structural integrity.

E.5 Seed-Dependent Convergence in Exec-Only GRPO

Not all exec-only seeds converge. Of 6 seeds tested (s42–s46, s52), we observe three regimes: *converging with degeneration* (s42: PR=93%, genuine=7.5%; s44: PR=96%, genuine=0%; s46: PR=93% at 500 steps), *slow convergence with degeneration* (s43: PR=14% but 100% hacking at 500 steps), and *non-converging* (s45: PR=15%, genuine=86.7%; s52: PR=14%, genuine=85.7%). Non-converging seeds retain high genuine rates (85–87%), indicating that structural degeneration is conditional on successful RL optimization: when GRPO fails to bootstrap prove rate beyond base-model levels, the model preserves structural integrity. Extended training to 1000 steps for s45 confirms persistent non-convergence (PR=15%, ISR*=0.592, genuine=86.7%, n=15 proved); the model neither improves nor degenerates.

This parallels the LLaMA-3.2-3B finding (Appendix E.4), where GRPO also fails to improve PR and retains structural quality. Both cases support the interpretation that degeneration is a byproduct of reward optimization success, not of the training procedure per se.

E.6 Cross-Scale: Qwen3-8B Exec-Only GRPO

To test whether structural degeneration generalizes beyond 4B scale, we apply exec-only GRPO to Qwen3-8B with K=8 rollouts per prompt (K=4 produces near-identical completions at 8B scale, yielding zero GRPO advantage and no learning signal). Table 12 summarizes configuration differences and results.

	Qwen3-4B	**Qwen3-8B**
Rollouts (K)	4	8
Batch size	4	1
Gradient accum.	2	16
Effective batch	8	16

Config	**PR%**	**Genuine%**	**Converged?**	**Gradient**
4B K=4 s42	93	7.5	Yes (degenerate)	Sufficient
8B K=4 s42	15.5	—	No	Zero
8B K=8 s42	14	92.9	No	Partial

Table 12: Cross-scale: Qwen3-4B vs. Qwen3-8B. **Top**: Configuration; K=8 required at 8B because K=4 produces near-identical rollouts. **Bottom**: Results; 4B converges with degeneration, both 8B configurations fail to converge, preserving structural integrity.

We first tested K=4 (matching the 4B configuration): the 8B model produces near-identical rollouts across all four completions, yielding zero GRPO advantage throughout 500 steps and no learning signal (PR=15.5%, cold-start stagnation). Increasing to K=8 provides non-zero reward variance during mid-training (approximately 40–50% of steps between 20–370 show non-zero advantage), but this signal proves insufficient for convergence: held-out evaluation yields PR=14% (n=14 proved), ISR*=0.573, genuine=92.9% (13/14), with a single CE sample. Cosine learning-rate decay combined with insufficient early bootstrap appears to prevent the model from escaping the low-PR regime.

This result mirrors two other non-convergence cases: (1) the seed-dependent 4B non-convergence (s45, s52; Appendix E.5), where identical 4B configurations fail to bootstrap, and (2) the LLaMA-3.2-3B cold-start failure (Appendix E.4), where the base model lacks FOL generation capability entirely. Together, these cases establish a convergence

threshold for GRPO-driven degeneration: the structural collapse documented in §4.2 requires successful RL optimization as a prerequisite, and at 8B scale the rollout diversity bottleneck prevents this optimization from occurring under our configuration.